T3000 API & Protocol

Message 9: SAVE_IMAGE · built Nov 13, 2026

Message 9: SAVE_IMAGE

The SAVE_IMAGE message uploads graphic image files from the web interface to the T3000 server for use in HMI graphics screens.

When to Use:

- Uploading background images for graphics
- Adding icon graphics
- Importing custom images for HMI screens

Overview

Action: `SAVE_IMAGE` (9) **Direction:** Frontend → Backend **Location:** BacnetWebView.cpp line 2855

Purpose: Upload and save image files to server uploads folder

Request Format

JSON Structure:

JSON

```
{
  "action": "SAVE_IMAGE",
  "filename": "floor-plan.png",
  "fileLength": 45820,
  "fileData": "iVBORw0KGgoAAAANSUhEUgAA..."
}
```

Request Parameters

Parameter	Type	Required	Description
`filename`	string	Yes	Image filename with extension
`fileLength`	number	Yes	File size in bytes
`fileData`	string	Yes	Base64-encoded image data

Supported File Types

Extension	Format	Common Use
.png	PNG	Icons, graphics with transparency
.jpg/.jpeg	JPEG	Photos, backgrounds
.gif	GIF	Simple animations
.bmp	Bitmap	Legacy graphics

Response Format

Success Response:

JSON

```
{
  "action": "SAVE_IMAGE_RES",
  "data": {
    "name": "floor-plan.png",
    "path": "/uploads/floor-plan.png"
  }
}
```

Error Response:

JSON

```
{
  "action": "SAVE_IMAGE_RES",
  "error": "Invalid file format"
}
```

Implementation

Location: BacnetWebView.cpp line 2855

C++

```cpp
case SAVE_IMAGE:
{
    const std::string filename = json.get("filename", Json::nullValue).asString();
    int file_length = json.get("fileLength", Json::nullValue).asInt();
    const std::string file_data = Json::writeString(builder, json["fileData"]);

    // Ensure uploads directory exists
    CString web_image_folder = g_strExePth + _T("ResourceFile\\webview\\www\\uploads");

    int ret = FALSE;
    WIN32_FIND_DATA fd;
    HANDLE hFind_folder = FindFirstFile(web_image_folder, &fd);
    if ((hFind_folder != INVALID_HANDLE_VALUE) &&
        (fd.dwFileAttributes & FILE_ATTRIBUTE_DIRECTORY))
    {
        ret = TRUE;
    }
    FindClose(hFind_folder);

    // Create directory if not exists
    if (ret == false)
    {
        SECURITY_ATTRIBUTES attrib;
        attrib.bInheritHandle = FALSE;
        attrib.lpSecurityDescriptor = NULL;
        attrib.nLength = sizeof(SECURITY_ATTRIBUTES);
        CreateDirectory(web_image_folder, &attrib);
    }

    // Handle duplicate filenames
    CString cs_filename = filename.c_str();
    CStringArray temp_array;
    CString temp_file_name_without_suffix;
    CString temp_suffix;
    SplitCStringA(temp_array, cs_filename, _T("."));

    if (temp_array.GetSize() >= 2)
    {
        CString temp_last = temp_array.GetAt(temp_array.GetSize() - 1);
        int temp_length = temp_last.GetLength();
```

```
        int total_length = cs_filename.GetLength();
        temp_file_name_without_suffix = cs_filename.Left(total_length - temp_length - 1)
        temp_suffix = temp_last;
    }
    else
    {
        break;  // Invalid filename (no extension)
    }

    // Generate unique filename if duplicate exists
    CString newfilename = cs_filename;
    int ncount = 1;
    while (PathFileExists(web_image_folder + _T("\\") + newfilename))
    {
        newfilename.Format(_T("%s(%d).%s"),
            temp_file_name_without_suffix, ncount++, temp_suffix);
    }

    // Decode base64 and write file
    CFile file;
    file.Open(web_image_folder + _T("\\") + newfilename,
        CFile::modeCreate | CFile::modeWrite, NULL);

    char* ret_result = (char*)malloc(file_length);
    base64_decode((char const*)temp_image_data.c_str(), ret_result,
        (int)strlen(temp_image_data.c_str()));
    file.write(ret_result, file_length);
    free(ret_result);

    // Send response
    tempjson["action"] = "SAVE_IMAGE_RES";
    tempjson["data"]["name"] = filename;
    tempjson["data"]["path"] = "/uploads/" + newfilename;

    const std::string output = Json::writeString(builder, tempjson);
    outmsg = CString(output.c_str());
}
```

File Storage

Upload Directory:

```
TEXT
```

```
[T3000 Install Path]\ResourceFile\webview\www\uploads\
```

File Access:

- Web path: `/uploads/filename.png`
- Full URL: `http://localhost:3003/uploads/filename.png`

Frontend Usage

Upload Image File

TYPESCRIPT

```typescript
const uploadImage = async (file: File): Promise<string> => {
  return new Promise((resolve, reject) => {
    const reader = new FileReader();

    reader.onload = () => {
      const base64Data = reader.result as string;
      // Remove data URL prefix (e.g., "")
      const base64Content = base64Data.split(',')[1];

      const message = {
        action: 'SAVE_IMAGE',
        filename: file.name,
        fileLength: file.size,
        fileData: base64Content
      };

      sendMessage(JSON.stringify(message));

      // Wait for response
      const handler = (data: string) => {
        const response = JSON.parse(data);

        if (response.action === 'SAVE_IMAGE_RES') {
          if (response.error) {
            reject(new Error(response.error));
          } else {
            resolve(response.data.path);
          }

          webSocket.off('message', handler);
        }
      };

      webSocket.on('message', handler);
    };
```

```
    reader.onerror = () => reject(new Error('Failed to read file'));
    reader.readAsDataURL(file);
  });
};

// Usage
const path = await uploadImage(imageFile);
console.log('Image uploaded to:', path);
```

Image Upload Component

TYPESCRIPT

```typescript
const ImageUploader: React.FC = () => {
  const [uploading, setUploading] = useState(false);
  const [uploadedPath, setUploadedPath] = useState<string | null>(null);

  const handleFileSelect = async (event: React.ChangeEvent<HTMLInputElement>) => {
    const file = event.target.files?.[0];
    if (!file) return;

    // Validate file type
    const validTypes = ['image/png', 'image/jpeg', 'image/gif', 'image/bmp'];
    if (!validTypes.includes(file.type)) {
      alert('Invalid file type. Please select an image file.');
      return;
    }

    // Validate file size (e.g., max 5MB)
    const maxSize = 5 * 1024 * 1024;
    if (file.size > maxSize) {
      alert('File too large. Maximum size is 5MB.');
      return;
    }

    try {
      setUploading(true);
      const path = await uploadImage(file);
      setUploadedPath(path);
      alert('Image uploaded successfully!');
    } catch (error) {
      alert(`Upload failed: ${error.message}`);
    } finally {
      setUploading(false);
    }
  };

  return (
    <div>
      <input
        type="file"
```

```
        accept="image/*"
        onChange={handleFileSelect}
        disabled={uploading}
      />
      {uploading && <span>Uploading...</span>}
      {uploadedPath && (
        <div>
          <p>Uploaded: {uploadedPath}</p>
          <img src={uploadedPath} alt="Uploaded" style={{ maxWidth: 300 }} />
        </div>
      )}
    </div>
  );
};
```

Duplicate Handling

If a file with the same name exists, the system automatically appends a number:

TEXT

```
Original: floor-plan.png
Exists, so saves as: floor-plan(1).png
Exists, so saves as: floor-plan(2).png
...
```

Validation

Client-Side Validation

TYPESCRIPT

```typescript
const validateImage = (file: File): { valid: boolean; error?: string } => {
  // Check file type
  const validTypes = ['image/png', 'image/jpeg', 'image/jpg', 'image/gif', 'image/bmp'];
  const fileType = file.type.toLowerCase();

  if (!validTypes.includes(fileType)) {
    return {
      valid: false,
      error: 'Invalid file type. Supported: PNG, JPEG, GIF, BMP'
    };
  }

  // Check file size (max 10MB)
  const maxSize = 10 * 1024 * 1024;
  if (file.size > maxSize) {
    return {
      valid: false,
      error: `File too large (${(file.size / 1024 / 1024).toFixed(2)}MB). Max: 10MB`
    };
  }

  // Check filename
  if (!/^[a-zA-Z0-9_\-\.]+$/.test(file.name)) {
    return {
      valid: false,
      error: 'Invalid filename. Use only letters, numbers, dashes, underscores.'
    };
  }

  return { valid: true };
};

// Usage
const validation = validateImage(file);
if (!validation.valid) {
  alert(validation.error);
```

```
    return;
}
```

Performance

Upload Times (approximate):

- Small icon (10KB): < 100ms
- Medium image (500KB): 200-500ms
- Large image (5MB): 1-3 seconds

Recommendations:

- Show upload progress indicator
- Optimize images before upload (compress, resize)
- Use web-optimized formats (PNG for graphics, JPEG for photos)

Error Handling

Common Errors

Error	Cause	Solution
No file extension	Invalid filename	Ensure filename has extension
Directory creation failed	Permission denied	Run T3000 as administrator
Base64 decode failed	Corrupted data	Re-encode file data
File write failed	Disk full	Free up disk space

Retry Logic

TYPESCRIPT

```typescript
const uploadWithRetry = async (
  file: File,
  maxRetries: number = 3
): Promise<string> => {
  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    try {
      return await uploadImage(file);
    } catch (error) {
      if (attempt === maxRetries) {
        throw error;
      }

      // Wait before retry (exponential backoff)
      await new Promise(resolve => setTimeout(resolve, 1000 * attempt));
    }
  }

  throw new Error('Upload failed after retries');
};
```

See Also

- DELETE_IMAGE - Deleting uploaded images
- SAVE_GRAPHIC_DATA - Using images in graphics
- Platform Overview - Architecture